Classificazione di uova parassitarie

Una soluzione completa di Machine Learning per immagini microscopiche acquisite in condizioni reali

AI-KFM / Kubic FLOTAC Microscope • Kaggle 2025–2026

Università degli Studi di Napoli Federico II

Corso: Algorithms, Data Structures and Machine Learning

97,61%
Balanced accuracy • test interno

97,53%
Accuracy • test interno

I punteggi sono stati ottenuti su **2.513 immagini etichettate**, riservate come insieme di test interno nel notebook; **non** sono presentati come risultati della leaderboard privata di Kaggle.

OBIETTIVO
Classificazione di immagini in 10 classi

DATI
15.701 patch microscopiche

ARCHITETTURA DI BASE
ConvNeXt-Tiny pre-addestrata su ImageNet

INFERENZA FINALE
Ensemble pesato di 3 modelli + TTA a 5 viste

Competenze principali

Computer Vision **Deep Learning** **PyTorch** **Progettazione esperimenti** **Transfer Learning** **Classi sbilanciate** **Ensemble**

Competizione: competizione su Kaggle • **Anno accademico:** 2025/26

Sintesi del progetto

Questa relazione descrive la progettazione, la sperimentazione e la valutazione di una pipeline di *Computer Vision* sviluppata per la *UniNa ML 25/26 Parasite Egg Classification Contest*. L'obiettivo era classificare dieci categorie di elementi parassitari contenuti in patch microscopiche di 224×224 pixel, acquisite mediante il Kubic FLOTAC Microscope (KFM), un sistema concepito a supporto delle attività di parassitologia veterinaria. A differenza dei comuni dataset di riferimento, le immagini fornite possono contenere detriti, illuminazione irregolare, oggetti sfocati e specie morfologicamente molto simili.[1,2]

Il progetto integra **transfer learning supervisionato**, **suddivisione stratificata dei dati**, **strategie di ottimizzazione per classi sbilanciate** e **confronto sistematico fra architetture e iperparametri**. Sono state addestrate sette configurazioni, distribuite su tre fold di validazione selezionati, utilizzando ConvNeXt-Tiny ed EfficientNetV2-S. Il sistema finale combina i migliori checkpoint ConvNeXt di ciascuno dei tre fold mediante un ensemble pesato sulle probabilità e una *Test-Time Augmentation* (TTA) basata su cinque trasformazioni geometriche.

> **Risultato verificato.** Su un insieme di test interno stratificato di 2.513 immagini, l'ensemble finale raggiunge una **balanced accuracy del 97,61%**, una **accuracy del 97,53%** e un **macro F1-score di circa 0,976**. Il notebook eseguito genera inoltre un file CSV di submission, verificato nella struttura, contenente 3.138 predizioni per il test set di Kaggle. Nei materiali disponibili non compare il punteggio della leaderboard privata.

Competenze messe in evidenza. Il lavoro dimostra la capacità di trasformare un dataset applicativo non ideale in un processo sperimentale riproducibile, scegliere i modelli sulla base di una metrica pertinente, interpretare i risultati delle soluzioni alternative e realizzare l'intera pipeline di inferenza e submission. Le sezioni successive documentano le operazioni effettivamente svolte, distinguendo le misure sperimentali dalle possibili evoluzioni future.

1. Definizione del problema e requisiti della challenge

1.1 Un problema di immagini clinico-veterinarie, non un benchmark semplificato

La diagnosi delle parassitosi in ambito veterinario si basa frequentemente sull'ispezione manuale dei campioni e su tecniche di conteggio delle uova, che richiedono tempo e possono limitare uniformità e scalabilità delle analisi. Il KFM è progettato per operare con le tecniche FLOTAC e Mini-FLOTAC, eseguire scansioni automatiche ed elaborare i dati a bordo. Durante l'acquisizione vengono registrati fotogrammi delle camere di flottazione, poi suddivisi in patch contenenti gli oggetti di interesse.[1,2]

La classificazione è complessa perché la qualità delle immagini varia in funzione della preparazione del campione, delle condizioni ambientali e della messa a fuoco. Inoltre, per distinguere alcune specie occorrono caratteristiche morfologiche sottili: ad esempio, i gusci delle uova di *Fasciola hepatica* e *Calicophoron daubneyi* possono risultare molto simili dopo il trattamento dei campioni. Un buon classificatore deve quindi apprendere caratteristiche morfologiche discriminanti senza dipendere eccessivamente da orientamento, illuminazione o impurità dello sfondo.

1.2 Dataset e formato delle predizioni

La challenge AI-KFM mette a disposizione **12.563 patch di addestramento etichettate** e **3.138 patch di test prive di etichetta**, per un totale di **15.701 immagini** conteggiate nel notebook. Ogni immagine misura 224×224 pixel. I metadati di addestramento comprendono il percorso dell'immagine, l'etichetta di classe, l'identificativo della sessione di scansione, l'ospite del campione, l'apparato e la data della sessione. Gli ID ufficiali delle dieci classi non sono consecutivi: il codice li converte pertanto negli indici $0, \ldots, 9$ durante l'addestramento, ripristinando gli ID originali prima della submission.

Tabella 1: Classi e identificativi ufficiali, mantenuti nella submission finale.

ID	Classe	ID	Classe
1	Strongili gastrointestinali	15	Strongyloides
3	*Fasciola hepatica*	17	*Moniezia benedeni*
5	*Calicophoron daubneyi*	20	*Ascaris lumbricoides*
7	Nematodirus	21	*Ancylostoma duodenale*
8	Bolla (Bubble)	22	*Trichuris trichiuria*

La metrica da massimizzare è la *balanced accuracy*, cioè la media aritmetica dei recall calcolati sulle singole classi:

$$\mathrm{BA} = \frac{1}{K}\sum_{k=1}^{K}\frac{TP_k}{TP_k + FN_k}, \qquad K = 10. \tag{1}$$

Questa scelta è importante perché riconoscere correttamente le classi più numerose non basta quando anche le categorie rare hanno interesse diagnostico. La submission richiesta è un CSV con due colonne, `ID` e `class_label`, che associa a ogni immagine del test uno degli identificativi ammessi.[1]

2. Preparazione dei dati e strategia di validazione

2.1 Suddivisione stratificata e insieme di test interno

Prima dell'addestramento è stato separato un insieme indipendente destinato alla valutazione interna. A tale scopo è stato utilizzato `StratifiedKFold` con cinque suddivisioni, mescolamento dei dati e seme casuale 99, selezionando il fold 0 come test interno. Le immagini etichettate rimanenti costituiscono l'insieme di sviluppo. Su quest'ultimo è stato applicato un secondo `StratifiedKFold`, con quattro suddivisioni e seme 50, per costruire coppie di training e validation: sono stati addestrati i modelli corrispondenti ai fold 3, 1 e 2. Si tratta quindi di un **ensemble costruito su tre fold selezionati**, non di una media esaustiva delle prestazioni di una cross-validation a quattro fold.

12.563 immagini etichettate $\longrightarrow$ **10.050** immagini di sviluppo $+$ **2.513** immagini di test interno

In ogni fold di sviluppo selezionato: circa 7.537–7.538 immagini per l'addestramento e 2.512–2.513 per la validazione.

Nel notebook eseguito, il test interno non viene impiegato per addestrare i checkpoint. Questa separazione lo rende più informativo di una valutazione eseguita sugli stessi dati usati per selezionare i modelli. Tuttavia, la stratificazione è stata effettuata in base alla *classe dell'immagine*, non alla sessione di scansione, nonostante il dataset contenga i relativi identificativi. La possibile somiglianza fra immagini della stessa sessione rappresenta quindi un limite metodologico, discusso nella Sezione 7.

2.2 Preprocessing e data augmentation

La pipeline carica le immagini RGB solo al momento dell'utilizzo, attraverso una classe `Dataset` personalizzata in PyTorch; i `DataLoader` separano le fasi di addestramento, validazione e test. In training, le immagini vengono ridimensionate a 224×224 pixel e sottoposte a ribaltamenti casuali orizzontali e verticali, ciascuno con probabilità 0,5, rotazioni fino a $\pm 20°$ e moderate variazioni di luminosità, contrasto e saturazione. Sia in training sia in inferenza si applica la normalizzazione con media e deviazione standard di ImageNet. La validazione *non* prevede trasformazioni casuali.

Queste operazioni sono motivate dalle caratteristiche delle acquisizioni microscopiche: l'orientamento degli oggetti può cambiare e l'illuminazione non è sempre costante. Le trasformazioni aiutano il modello

a ridurre la dipendenza da proprietà accidentali della ripresa e a concentrarsi sugli elementi morfologici utili alla classificazione. L'implementazione prevede batch di 32 immagini, due worker per il caricamento, `pin_memory=True` e worker persistenti, per limitare il costo del trasferimento dei dati dalla CPU alla GPU.

3. Architetture e strategia di addestramento

3.1 Transfer learning con ConvNeXt-Tiny

L'architettura principale è **ConvNeXt-Tiny**, inizializzata con pesi pre-addestrati su ImageNet mediante `torchvision`. Il classificatore originario è stato sostituito con uno strato `Dropout(p=0.20)` e una nuova proiezione lineare verso le dieci classi del problema. In questo modo si riutilizzano le rappresentazioni visive apprese su un dataset generale, adattando il classificatore finale alle categorie parassitarie. Come confronto architetturale è stata sperimentata anche **EfficientNetV2-S**, anch'essa pre-addestrata su ImageNet, senza presupporre che fosse necessariamente superiore.

3.2 Ottimizzazione per classi sbilanciate

La funzione obiettivo è una *cross-entropy pesata* con *label smoothing* pari a $\varepsilon = 0{,}05$. Indicando con n_k il numero di esempi della classe k nel training set, i pesi sono calcolati nel modo seguente:

$$\widetilde{w}_k = \frac{1}{\sqrt{n_k}}, \qquad w_k = \frac{\widetilde{w}_k}{\frac{1}{K}\sum_{j=1}^{K} \widetilde{w}_j}. \tag{2}$$

Il peso inversamente proporzionale alla radice quadrata della numerosità attribuisce maggiore importanza alle classi rare, senza amplificarne il contributo quanto un peso pienamente inverso alla frequenza. Il label smoothing attenua l'eccessiva sicurezza delle etichette target, mentre dropout, *weight decay* e data augmentation contribuiscono alla regolarizzazione. L'ottimizzatore è AdamW, con weight decay 10^{-4}, learning rate regolati mediante *cosine annealing*, clipping della norma del gradiente a 1,0 e *automatic mixed precision* su CUDA (FP16 con gradient scaling).

3.3 Fine-tuning in due fasi e configurazioni sperimentali

Ogni esperimento inizia con **due epoche di addestramento della sola testa di classificazione**, mantenendo congelato il backbone. Successivamente, il backbone viene sbloccato per il fine-tuning completo, con learning rate distinti per estrattore di caratteristiche e classificatore. Sono state confrontate due configurazioni:

Tabella 2: Configurazioni di ottimizzazione effettivamente implementate nel notebook.

Configurazione	Epoche totali	LR testa iniziale	LR backbone	LR testa nel fine-tuning
LR più alto	12	1×10^{-3}	2×10^{-5}	1×10^{-4}
LR più basso	18	5×10^{-4}	1×10^{-5}	5×10^{-5}

Gli esperimenti sono stati eseguiti nell'ambiente Python di Kaggle su una GPU **NVIDIA Tesla P100 da 16 GB**. Il notebook riporta PyTorch 2.10.0 con librerie CUDA 12.6. Sono stati fissati i semi casuali per Python, NumPy e PyTorch, senza tuttavia imporre kernel completamente deterministici: la riproducibilità esatta, bit per bit, non è pertanto garantita fra sistemi differenti. Per la maggior parte degli esperimenti sono stati salvati il miglior checkpoint di validazione e lo storico delle metriche epoca per epoca.

4. Confronto degli esperimenti e selezione dei modelli

Il criterio di selezione è stato il valore massimo di *balanced accuracy sul validation set* ottenuto da ciascuna configurazione. La Tabella 3 riporta i sette addestramenti completati e documentati dagli output del notebook, comprese le alternative non scelte per l'ensemble finale.

Tabella 3: Sette esperimenti eseguiti. In grassetto i checkpoint inclusi nell'ensemble finale.

Architettura / fold	Configurazione	Epoche	Epoca migliore	Miglior BA val.
ConvNeXt-Tiny / fold 3	**LR più alto**	12	11	**97,71%**
ConvNeXt-Tiny / fold 3	LR più basso	18	16	97,24%
ConvNeXt-Tiny / fold 1	**LR più alto**	12	9	**97,31%**
ConvNeXt-Tiny / fold 1	LR più basso	18	15	96,96%
ConvNeXt-Tiny / fold 2	LR più basso	18	16	97,28%
ConvNeXt-Tiny / fold 2	**LR più alto**	12	10	**97,41%**
EfficientNetV2-S / fold 3	LR più basso	18	18	96,57%

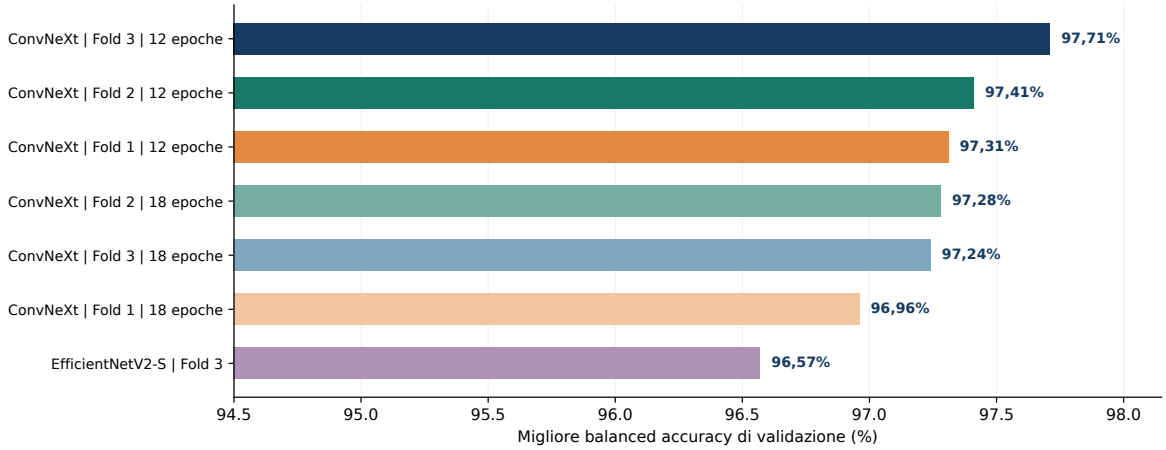


Figura 1: Miglior balanced accuracy di validazione nei sette esperimenti. L'asse orizzontale è volutamente troncato per evidenziare le differenze nella fascia di prestazioni elevate; i valori sono riportati nella Tabella 3.

Nei fold 3 e 1, la configurazione di 12 epoche supera quella di 18 epoche. Anche nel fold 2, il miglior checkpoint del training più breve ottiene un risultato leggermente superiore. EfficientNetV2-S costituisce una soluzione di confronto credibile, ma la sua prestazione massima nel fold 3 è inferiore a quelle ottenute dalle due configurazioni ConvNeXt sul medesimo fold. La scelta finale privilegia quindi la qualità osservata in validazione rispetto alla novità dell'architettura o alla durata dell'addestramento.

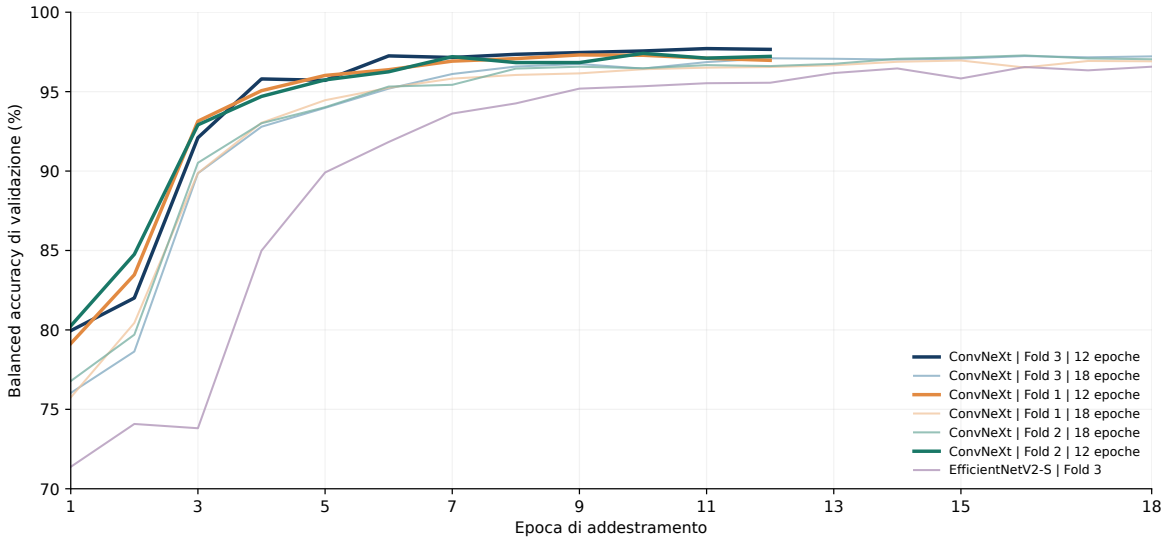


Figura 2: Andamento della balanced accuracy di validazione, ricostruito dagli output epoca per epoca del notebook Kaggle. Il cambiamento osservabile dopo la seconda epoca coincide con lo sblocco del backbone.

5. Ensemble finale e submission su Kaggle

5.1 Selezione dei checkpoint effettivamente utilizzati

Per ciascuno dei fold selezionati è stato caricato il checkpoint compatibile con il miglior risultato di validazione. La composizione finale, verificata negli output di caricamento dei checkpoint, è la seguente: **fold 3 / 12 epoche: 97,71%** con peso 0,50; **fold 1 / 12 epoche: 97,31%** con peso 0,25; **fold 2 / 12 epoche: 97,41%** con peso 0,25. Questo dato chiarisce un commento Markdown non aggiornato del notebook, nel quale compariva il modello del fold 3 addestrato per 18 epoche: la logica di selezione eseguita e gli output dell'inferenza mostrano invece l'utilizzo del modello a 12 epoche.

5.2 Media delle probabilità su modelli e trasformazioni

Ciascuno dei tre classificatori viene valutato su cinque viste deterministiche: immagine originale, ribaltamento orizzontale, ribaltamento verticale, rotazione di $90°$ e rotazione di $270°$. L'implementazione calcola dapprima le probabilità mediante `softmax`, ne effettua la media sulle viste e infine combina i tre modelli attraverso una media pesata:

$$\widehat{\mathbf{p}}(x) = \sum_{m=1}^{3} \alpha_m \left[\frac{1}{5} \sum_{t \in \mathcal{T}} \mathrm{softmax}\big(f_m(t(x))\big) \right], \quad \boldsymbol{\alpha} = (0{,}50, 0{,}25, 0{,}25), \tag{3}$$

$$\widehat{y}(x) = \mathrm{argmax}_{k \in \{0,\dots,9\}} \widehat{p}_k(x), \qquad \mathcal{T} = \{I, H, V, R_{90}, R_{270}\}. \tag{4}$$

Una mappatura inversa riporta la classe predetta $\widehat{y}$ all'identificativo ufficiale della competizione. La configurazione richiede 15 passaggi *forward* per immagine (tre modelli e cinque viste), accettando un costo di inferenza più elevato in cambio dell'aggregazione delle predizioni. Poiché nel notebook non è presente un confronto controllato fra inferenza *senza TTA* e inferenza *con TTA* sul medesimo test interno, non viene attribuito alla sola TTA alcun miglioramento numerico specifico.

5.3 Generazione e verifica del file di submission

Il dataset di inferenza estrae gli ID numerici da file denominati `test_patch_<ID>.jpg`, li ordina e verifica la corrispondenza con gli ID presenti nel file di submission di riferimento. Le predizioni vengono quindi ordinate, riconvertite alle etichette originali e salvate in `submission_3modelli_tta5.csv`. Alcune

istruzioni di controllo verificano la presenza di **3.138 righe**, **3.138 ID univoci** e **nessuna etichetta predetta mancante**. Il CSV prodotto rispetta il formato a due colonne richiesto dalla challenge; il risultato sulla leaderboard privata non è riportato nel notebook fornito.

6. Valutazione sul test interno e interpretazione dei risultati

97,61% Balanced accuracy

97,53% Accuracy

0,976 Macro F1-score

Test interno: 2.513 immagini etichettate, dieci classi

Il macro recall coincide, salvo arrotondamenti, con la balanced accuracy, come previsto dall'Equazione (1). Il report per classe è particolarmente significativo: *Fasciola hepatica* è la categoria più difficile in termini di recall (94,6%), mentre *Calicophoron daubneyi* raggiunge il 96,3%. La difficoltà di distinguerle è coerente con il contesto applicativo, poiché i gusci delle rispettive uova possono essere molto simili dopo la preparazione. Altre classi, come Nematodirus (99,1%) e *Ascaris lumbricoides* (99,0%), mostrano recall più elevati su questa partizione.

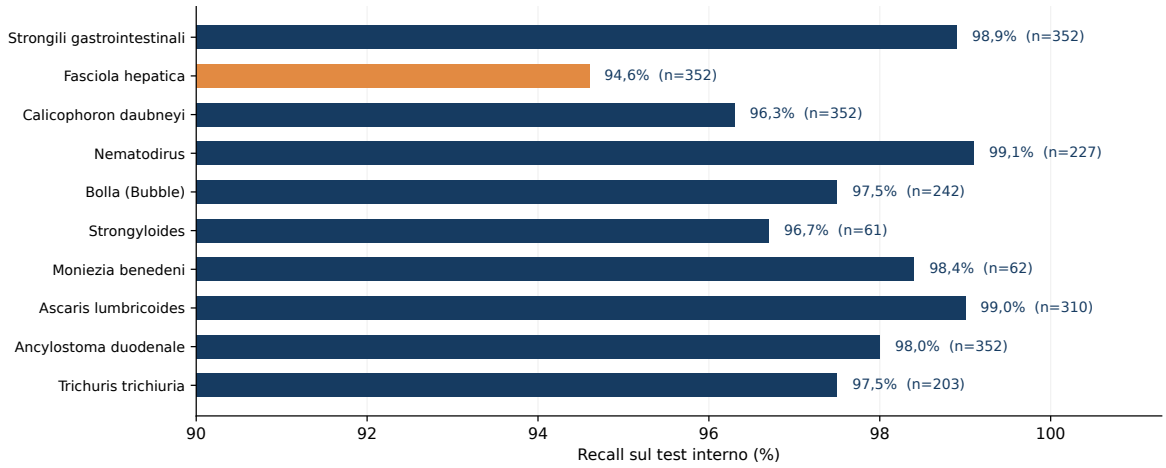


Figura 3: Recall per classe sul test interno, ricostruito dal classification report. I valori tra parentesi indicano il numero di esempi della classe; Strongyloides e Moniezia, per esempio, dispongono rispettivamente di 61 e 62 campioni.

La tabella seguente approfondisce le prestazioni delle due specie visivamente simili, rendendo esplicite differenze che un singolo indicatore aggregato potrebbe nascondere.

Tabella 4: Risultati dettagliati sul test interno per due classi particolarmente impegnative.

Classe	Precision	Recall	F1-score	Campioni
Fasciola hepatica	0,965	0,946	0,956	352
Calicophoron daubneyi	0,942	0,963	0,952	352

Questi valori derivano da misurazioni effettive sullo split interno del notebook, ma non costituiscono una validazione clinico-diagnostica del sistema. La loro attendibilità come stima delle prestazioni in esercizio dipende da ulteriori prove su nuove sessioni di scansione e differenti condizioni di acquisizione.

7. Competenze ingegneristiche e limiti metodologici

7.1 Competenze tecniche trasferibili

Impostazione del problema e scelta della metrica. È stata utilizzata la balanced accuracy per attribuire importanza anche alle categorie minoritarie; inoltre, le metriche per classe hanno fornito un quadro più informativo rispetto alla sola accuracy globale.

Sviluppo completo della pipeline di Deep Learning. L'implementazione comprende lettura delle immagini, codifica delle etichette, augmentation riservata al training, dataset e data loader personalizzati, adattamento di backbone pre-addestrati, ottimizzazione a precisione mista, salvataggio e caricamento dei checkpoint, inferenza con TTA e verifica automatizzata della submission.

Selezione dei modelli basata su evidenze. Il notebook confronta configurazioni con learning rate e durate differenti, oltre a un'architettura alternativa. La scelta dei checkpoint si fonda sulle prestazioni misurate in validazione, senza assumere che un addestramento più lungo o un learning rate più basso siano sempre migliori.

Analisi quantitativa dei risultati. Un test interno riservato, metriche di precision, recall e F1 per ogni classe e curve di apprendimento comparative consentono di esaminare sia le prestazioni aggregate sia le categorie più esposte a errori.

7.2 Che cosa i risultati *non* dimostrano

Indipendenza fra sessioni non garantita. Sebbene siano disponibili gli identificativi delle sessioni, gli split di validazione e test interno vengono costruiti stratificando le singole immagini, non i gruppi. Patch della stessa scansione potrebbero quindi comparire in partizioni differenti, rendendo potenzialmente ottimistica la stima delle prestazioni su sessioni completamente nuove. Un successivo studio dovrebbe utilizzare una suddivisione per sessioni disgiunte, possibilmente stratificata per classe, oppure valutare la separazione per ospite, controllando la rappresentatività delle categorie.

Esperimenti di ablazione non completi. Il sistema finale con ensemble e TTA è stato valutato, ma non è stato isolato quantitativamente il contributo di ensemble, TTA, pesatura delle classi e singole trasformazioni di data augmentation. Esperimenti controllati permetterebbero di stabilire quali componenti giustifichino il proprio costo computazionale.

Nessuna dichiarazione di impiego clinico. I materiali non riportano test su centri esterni, calibrazione dell'incertezza, latenza di inferenza sull'hardware KFM, idoneità regolatoria o punteggio della leaderboard privata di Kaggle. I risultati documentano una *pipeline avanzata sviluppata per una challenge universitaria*, non un dispositivo diagnostico veterinario già validato per l'uso operativo.

7.3 Possibili sviluppi

Il passo successivo più utile sarebbe uno **studio di generalizzazione su sessioni di scansione disgiunte**, accompagnato da un'analisi delle confusioni tra classi, in particolare tra *Fasciola* e *Calicophoron*. Sarebbe inoltre opportuno confrontare sullo stesso test interno un modello singolo, l'ensemble e le varianti con TTA, per valutare poi latenza e compressione dei modelli nell'ottica dell'esecuzione *edge*. La calibrazione delle probabilità e la possibilità di astenersi dalle predizioni a bassa confidenza potrebbero consentire di sottoporre i casi ambigui a un esperto invece di imporre una classificazione incerta.

8. Competenze dimostrate

Metodologie: classificazione di immagini, transfer learning con CNN, fine-tuning, trattamento dello sbilanciamento tra classi, validazione stratificata, confronto sperimentale, scelta dei checkpoint, ensemble learning, test-time augmentation e analisi delle metriche per classe.

Strumenti: Python, PyTorch, torchvision, scikit-learn, NumPy, pandas, Pillow, Matplotlib, Kaggle Notebooks e addestramento GPU CUDA/FP16.

Aspetti ingegneristici: controllo dei semi casuali, pipeline di caricamento in batch, gestione delle risorse GPU, registrazione automatica degli storici di training, serializzazione dei modelli, decodifica coerente delle etichette e validazione della submission mediante `assert`.

Considerazione conclusiva. Questo progetto va oltre il semplice addestramento di una rete pre-addestrata: documenta un processo iterativo di Machine Learning che parte dai vincoli dell'acquisizione, seleziona metriche coerenti con il problema, costruisce un addestramento robusto, confronta soluzioni alternative, interpreta gli errori per classe e produce un artefatto di inferenza verificato. Il miglior risultato dimostrato è una **balanced accuracy del 97,61% sul test interno**. L'esame universitario complessivo è stato superato con **30/30**, secondo quanto comunicato dall'autore: questo voto accademico è distinto dal punteggio della competizione.

Appendice tecnica: evidenze e riproducibilità

Risultati completi della classificazione sul test interno

La tabella seguente riporta le metriche per classe mostrate nell'output della valutazione finale del notebook. Il recall è la grandezza utilizzata nel calcolo della balanced accuracy; il numero di campioni consente di interpretare le metriche anche in relazione alla disponibilità di esempi per ogni categoria.

Tabella 5: Metriche per classe sul test interno (2.513 campioni totali).

Classe	Precision	Recall	F1	Campioni
Strongili gastrointestinali	0,983	0,989	0,986	352
Fasciola hepatica	0,965	0,946	0,956	352
Calicophoron daubneyi	0,942	0,963	0,952	352
Nematodirus	0,996	0,991	0,993	227
Bolla (Bubble)	0,992	0,975	0,983	242
Strongyloides	0,952	0,967	0,959	61
Moniezia benedeni	1,000	0,984	0,992	62
Ascaris lumbricoides	0,978	0,990	0,984	310
Ancylostoma duodenale	0,986	0,980	0,983	352
Trichuris trichiuria	0,975	0,975	0,975	203
Media macro	**0,977**	**0,976**	**0,976**	**2.513**

Tracciabilità degli esperimenti e condizioni di replica

Il notebook conserva la configurazione del caricamento dati, i semi utilizzati per gli split di training e validation, i sette esperimenti eseguiti, la selezione dei checkpoint, il report finale sul test interno e la generazione della submission con 3.138 righe. Per ripetere gli esperimenti occorrono il dataset originale di Kaggle, i checkpoint pre-addestrati di ImageNet impiegati, una versione di PyTorch/torchvision compatibile con l'ambiente documentato e memoria GPU sufficiente. Gli esperimenti hanno utilizzato il seme 42 per Python, NumPy e PyTorch, nonché i semi 99 e 50 per le suddivisioni. La selezione dei checkpoint deve avvenire sui dati di validazione, senza ricorrere al test interno.

Origine dei risultati e riferimenti

Evidenze del notebook. Le scelte architetturali, gli iperparametri, le metriche per epoca, i risultati sul test interno e i dettagli della submission sono stati confrontati con il notebook eseguito fornito dall'autore (`notebookpulito (2).ipynb`). I grafici di validazione e confronto tra esperimenti sono stati ricostruiti dagli output registrati; il grafico dei recall è stato ricavato dal classification report. Per preparare questa relazione non sono stati ripetuti né gli addestramenti né le inferenze.

Fonti

[1] UniNa ML 25/26, *Parasite Egg Classification Contest*, descrizione ufficiale della competizione su Kaggle e requisiti del dataset (Università degli Studi di Napoli Federico II, a.a. 2025/26). competizione su Kaggle.

[2] C. Sansone e S. Capuozzo, *Machine Learning Contest: Parasite Egg Classification from Kubic FLOTAC Microscope Samples Images*, presentazione ufficiale di 21 slide, 15 maggio 2026 (PDF fornito dall'autore).

[3] S. Capuozzo *et al.*, "Automating parasite egg detection: insights from the first AI-KFM challenge", *Frontiers in Artificial Intelligence*, vol. 7, 2024. doi:10.3389/frai.2024.1325219.

Nota sulla pubblicazione: i grafici di questa relazione contengono metriche derivate, non immagini originali del dataset. Prima di distribuire pubblicamente patch microscopiche o file dei modelli, verificare le regole di utilizzo e la licenza della competizione.